UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

Facultatea de Automatică și Calculatoare
Specializarea: Calculatoare și Tehnologia Informației

Structura Sistemelor de Calcul

Program de testare a parametrilor de performanță ai unui PC

Coordonator Științific:
Mădălin Neagu

Student:
Czebely Stefania-Elena

Cluj-Napoca
2025

Cuprins

1 1. Introducere și Obiective 3

1.1 1.1. Argumentul Proiectului 3

1.2 1.2. Obiectivele Aplicației 3

2 2. Studiu Bibliografic și Teoretic 4

2.1 2.1. Arhitectura Procesorului (CPU) 4

2.2 2.2. Ierarhia de Memorie și Latența 4

2.3 2.3. Fractalii și Testarea Computațională 5

3 3. Analiza și Arhitectura Aplicației 6

3.1 3.1. Limbajul Python 6

3.2 3.2. Biblioteci Externe și Standard 6

3.3 3.3. Structura Modulară 6

4 4. Detalii de Implementare 8

4.1 4.1. Monitorizare Hardware 8

4.2 4.2. Testul de Memorie (Viteză și Latență) 8

4.3 4.3. Testele de Procesare (Integer și Float) 8

4.4 4.4. Calculul Scorului Final 9

5 5. Ghid de Utilizare și Interfața Grafică 10

5.1 5.1. Ecran Start (Interfață) 10

5.2 5.2. Informații Procesor 10

5.3 5.3. Informații Memorie 11

5.4 5.4. Test Timp Acces Memorie 12

5.5 5.5. Test Viteză Transfer Memorie 12

5.6 5.6. Test Operații Întregi 13

5.7 5.7. Test Operații Flotante 13

5.8 5.8. Test Mandelbrot 14

6 6. Concluzii și Evaluare 15

6.1 6.1. Analiza Performanței 15

6.2 6.2. Direcții de Dezvoltare 15
6.3 6.3. Concluzie Finală 16

A Anexa A: Codul Sursă al Aplicației **18**

1. Introducere și Obiective

1.1 1.1. Argumentul Proiectului

În contextul evoluției rapide a tehnologiei hardware, diversitatea componentelor (procesoare, memorii, plăci de bază) a crescut exponențial. Pentru un utilizator obișnuit sau un administrator de sistem, devine dificilă evaluarea obiectivă a performanței unui calculator doar citind specificațiile tehnice de pe cutie. Frecvența procesorului (GHz) nu mai este singurul indicator relevant, arhitectura internă, numărul de instrucțiuni pe ciclu (IPC) și viteza memoriei RAM jucând roluri cruciale.

Acest proiect propune o soluție software de tip *Benchmark* (testare sintetică), capabilă să streseze controlat componentele principale ale unui PC și să ofere un scor cuantificabil. Aplicația este scrisă în limbajul Python, având o interfață grafică prietenoasă, și a fost dezvoltată pentru a îndeplini cerința nr. 14: *„Program de testare a parametrilor de performanță ai unui PC”*.

1.2 1.2. Obiectivele Aplicației

Principalul obiectiv este crearea unui utilitar software care să ofere o imagine de ansamblu asupra sănătății și vitezei sistemului. Obiectivele specifice sunt:

1. **Identificarea Hardware Non-invazivă:** Extragerea automată a metadatelor despre procesor (Model, Arhitectură, Nuclee) și Memorie (Capacitate, Încărcare) folosind apeluri de sistem.

2. **Testarea Unității Aritmetico-Logice (ALU):** Măsurarea puterii brute de calcul prin operații cu numere întregi și operații logice pe biți.

3. **Testarea Unității de Virgulă Mobilă (FPU):** Evaluarea performanței în calcule științifice și grafice (floating point), inclusiv generarea de fractali.

4. **Testarea Subsistemului de Memorie:** Analiza lățimii de bandă (throughput) și a latenței (timpul de acces) pentru memoria RAM.

5. **Experiență Utilizator (UX):** Implementarea unei interfețe grafice (GUI) care să nu se blocheze în timpul testelor intensive, folosind programare concurentă (Threading).

2. Studiu Bibliografic și Teoretic

2.1 2.1. Arhitectura Procesorului (CPU)

Unitatea Centrală de Prelucrare este componenta care execută instrucțiunile unui program. Performanța sa este determinată de mai mulți factori:

- **Frecvența de ceas:** Numărul de cicluri pe care CPU-ul le poate executa într-o secundă.
- **Nuclee Fizice vs. Logice:** Procesoarele moderne folosesc tehnologii precum *Hyper-Threading* (Intel) sau *SMT* (AMD) pentru a permite unui singur nucleu fizic să execute două fire de execuție simultan. Aplicația noastră detectează ambele tipuri.
- **Setul de Instrucțiuni:** Testele sintetice (precum cel dezvoltat în acest proiect) utilizează instrucțiuni de bază (ADD, MUL, DIV) pentru a asigura compatibilitatea cu orice arhitectură x86_64.

2.2 2.2. Ierarhia de Memorie și Latența

Memoria unui calculator este organizată ierarhic. Procesorul are nevoie de date extrem de rapid.

1. **Registre:** Cea mai rapidă memorie, situată în interiorul CPU.
2. **Cache (L1, L2, L3):** Memorie SRAM foarte rapidă, dar scumpă și de capacitate mică.
3. **RAM (Random Access Memory):** Memorie DRAM, cu capacități mari (8GB - 64GB), dar mult mai lentă decât cache-ul.

În cadrul acestui proiect, testăm viteza memoriei RAM. Un aspect critic este diferența dintre **Accesul Secvențial** (predictibil, rapid, ajutat de mecanismele de pre-fetching ale CPU) și **Accesul Aleatoriu** (impredictibil, lent, cauzează deseori ”Cache Miss”).

2.3 2.3. Fractalii și Testarea Computațională

Pentru a testa stabilitatea sistemului în sarcină maximă, am ales algoritmul **Mandelbrot**. Acesta este un set de numere complexe c pentru care funcția $f_c(z) = z^2 + c$ nu diverge la infinit. Calculul acestui fractal necesită milioane de iterații și operații cu numere reale (float), fiind un test excelent pentru FPU (Floating Point Unit) și pentru generarea de căldură în procesor, verificând astfel și sistemul de răcire.

3. Analiza și Arhitectura Aplicației

3.1 3.1. Limbajul Python

Am ales Python datorită versatilității sale și a bibliotecilor puternice pentru interacțiunea cu sistemul de operare. Deși este un limbaj interpretat, modulele critice (precum `math` sau `time`) sunt optimizate în C, permițând măsurători precise.

3.2 3.2. Biblioteci Externe și Standard

Proiectul se bazează pe următoarele biblioteci:

- **Tkinter:** Biblioteca standard Python pentru crearea interfețelor grafice (GUI). Permite crearea de ferestre, butoane, etichete și zone de text.
- **Psutil (Process and System Utilities):** O bibliotecă esențială pentru monitorizare. Este folosită pentru a interoga nucleul sistemului de operare despre frecvența procesorului în timp real și gradul de utilizare a memoriei RAM.
- **Cpuinfo:** O bibliotecă specializată care parsează fișierul `/proc/cpuinfo` (pe Linux) sau registrul (pe Windows) pentru a obține numele comercial exact al procesorului (ex: ”Intel Core Ultra 5”).
- **Threading:** Utilizată pentru a rula testele de performanță pe fire de execuție separate de firul principal al interfeței grafice. Fără `threading`, fereastra aplicației ar îngheța (”Not Responding”) pe durata calculelor matematice intensive.

3.3 3.3. Structura Modulară

Aplicația este construită pe principiile Programării Orientate pe Obiect (OOP). Clasa principală, `PerformanceTestGUI`, gestionează:

1. **Layout-ul:** Împărțirea ferestrei în panoul de control (stânga) și panoul de rezultate (dreapta).

2. **Logica de Testare:** Metode dedicate pentru fiecare tip de test (`test_cpu_info`, `test_mandelbrot`, etc.).

3. **Sistemul de Scorare:** O metodă finală care agregă rezultatele și le compară cu un sistem de referință.

4. Detalii de Implementare

4.1 4.1. Monitorizare Hardware

Funcția de monitorizare interoghează sistemul la momentul apăsării butonului.

```
def test_cpu_info(self):
    info = cpuinfo.get_cpu_info()
    freq = psutil.cpu_freq()
    # Afisare nuclee fizice vs logice
    # Afisare frecventa curenta
```

Aceasta nu stresează sistemul, ci doar raportează configurația detectată.

4.2 4.2. Testul de Memorie (Viteză și Latență)

Pentru a măsura viteza, alocăm blocuri de date de dimensiuni variabile (1KB, 10KB, 100KB, 1MB) și le copiem repetat în RAM.

$$Viteza(MB/s) = \frac{DimensiuneBloc \times Iteratii}{Timp \times 1024^2} \tag{4.1}$$

Testul de latență compară parcurgerea unui vector index cu index $(i, i+1, ...)$ față de accesarea unor indici generați aleatoriu $(random(0, N))$. Diferența uriașă de timp observată (de ordinul nanosecundelor per acces) demonstrează penalizarea adusă de accesul la memoria principală RAM în loc de memoria Cache.

4.3 4.3. Testele de Procesare (Integer și Float)

Aceste teste execută bucle de 10 milioane de iterații.

- **Integer:** Adunări și înmulțiri simple. Măsoară performanța în sarcini generale.
- **Float:** Operații cu zecimale. Măsoară performanța FPU.
- **Mandelbrot:** Generează o matrice 400x400 de pixeli, calculând pentru fiecare pixel apartenența la mulțime. Este cel mai intensiv test din suită.

4.4 4.4. Calculul Scorului Final

Scorul este calculat ponderat, acordând prioritate puterii de procesare:

$$Scor = 0.25 \cdot S_{int} + 0.25 \cdot S_{float} + 0.25 \cdot S_{mandel} + 0.15 \cdot S_{mem} + 0.10 \cdot S_{logic} \tag{4.2}$$

5. Ghid de Utilizare și Interfața Grafică

Acest capitol detaliază pașii de utilizare a aplicației, prezentând funcționalitatea fiecărui buton în ordinea execuției.

5.1 5.1. Ecran Start (Interfață)

La pornirea aplicației, se afișează interfața grafică principală. Panoul din stânga conține lista testelor disponibile, iar panoul din dreapta este inițial gol, așteptând rezultatele.



Figura 5.1: Interfața inițială a aplicației (Ecran Start).

5.2 5.2. Informații Procesor

Utilizatorul apasă primul buton din meniu, „Informații Procesor”. Butonul devine verde pentru a indica succesul, iar în dreapta se afișează detaliile hardware detectate: modelul procesorului (Intel Core Ultra 5), arhitectura și frecvențele.

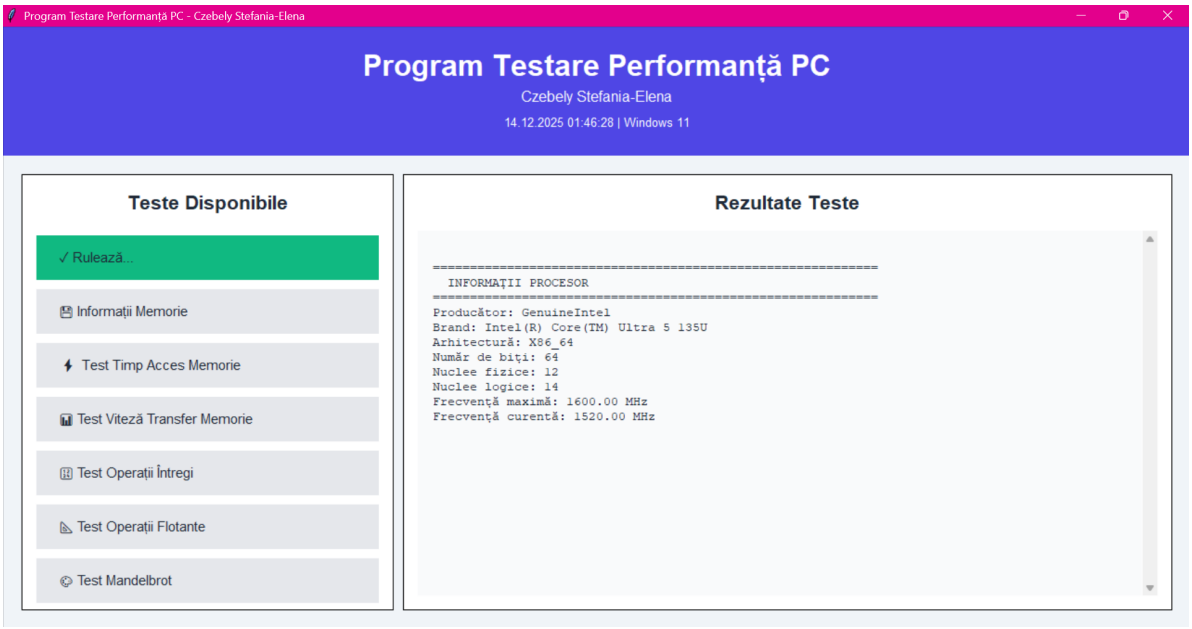


Figura 5.2: Rezultatele afișate după apăsarea butonului Informații Procesor.

5.3 5.3. Informații Memorie

La apăsarea celui de-al doilea buton, aplicația interoghează sistemul de operare și afișează cantitatea totală de memorie RAM, memoria disponibilă și procentul de utilizare curent.



Figura 5.3: Afișarea informațiilor despre Memorie sub cele de Procesor.

5.4 5.4. Test Timp Acces Memorie

Acest test măsoară latența. Se rulează algoritmul de acces secvențial versus cel aleatoriu. Diferența de timp (în nanosecunde) indică eficiența memoriei cache.

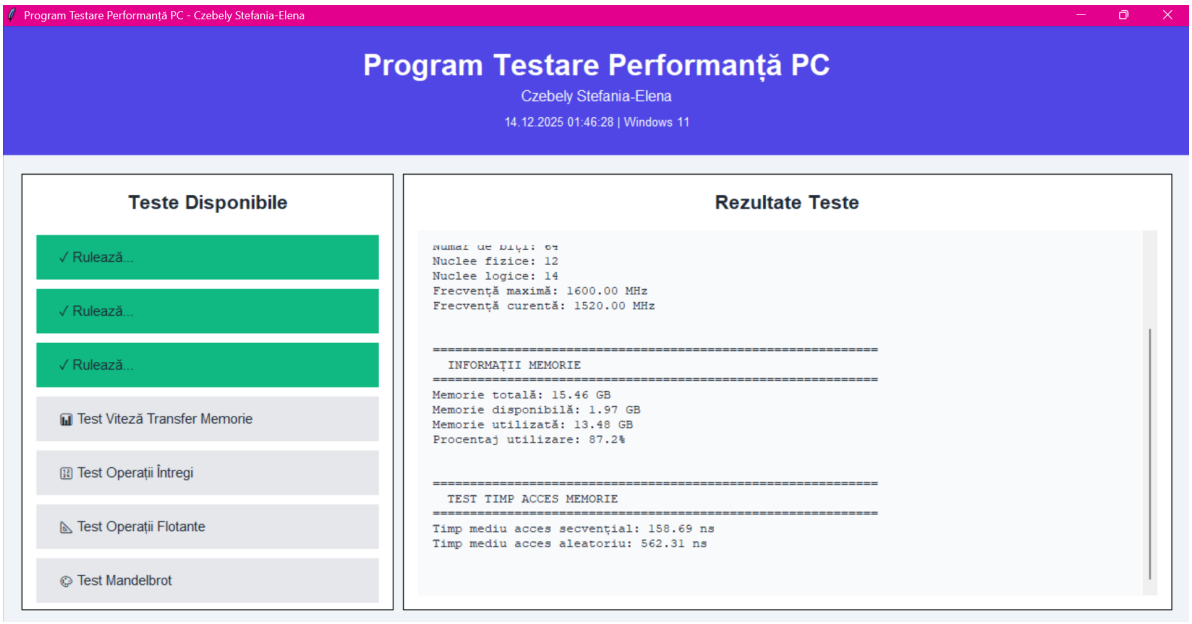


Figura 5.4: Rezultatele testului de Timp Acces Memorie (Latență).

5.5 5.5. Test Viteză Transfer Memorie

Se apasă butonul „Test Viteză Transfer Memorie”. Aplicația copiază blocuri de date de dimensiuni diferite și calculează lățimea de bandă (Throughput) în MB/s.

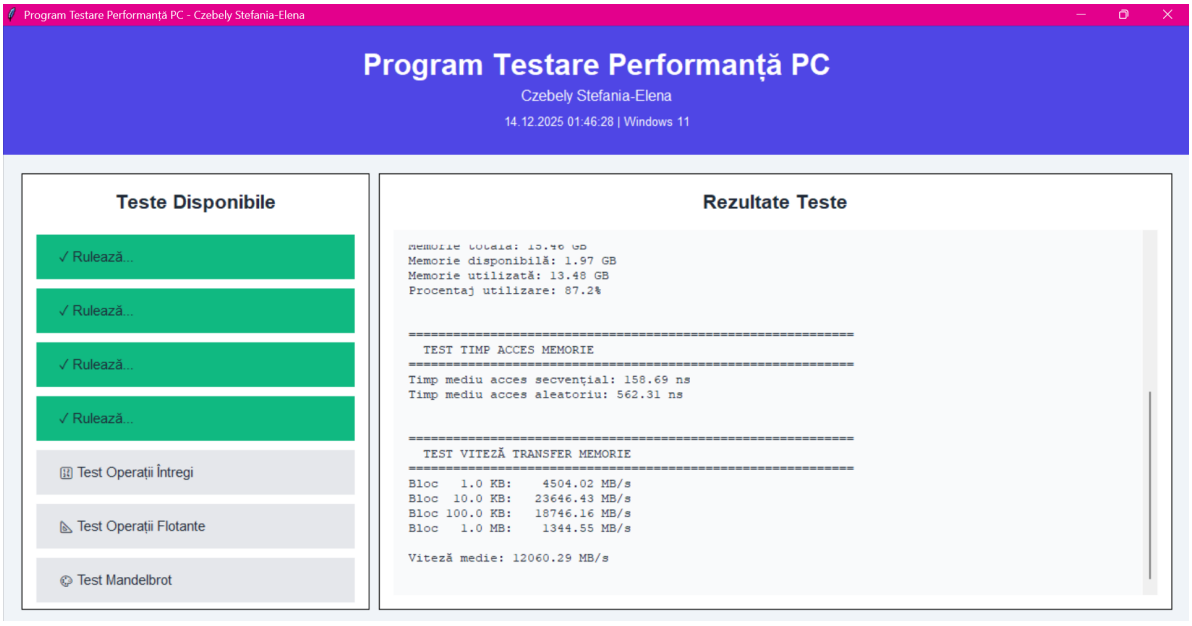


Figura 5.5: Rezultatele testului de viteză de transfer (peste 12.000 MB/s).

5.6 5.6. Test Operații Întregi

Testul de stres pentru ALU (Unitatea Aritmetică Logică). Se execută milioane de adunări și înmulțiri cu numere întregi, rezultatul fiind afișat în MOPS (Milioane Operații Pe Secundă).

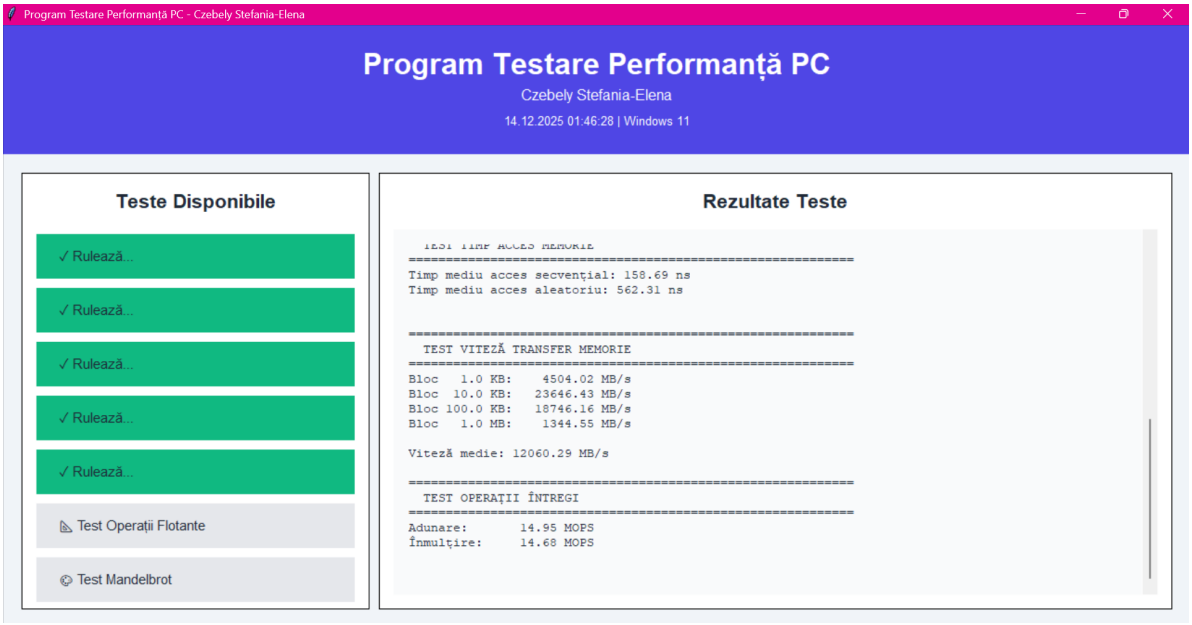


Figura 5.6: Rezultatele testului de Operații Întregi (sus).

5.7 5.7. Test Operații Flotante

Imediat sub rezultatele anterioare, se afișează performanța FPU (Floating Point Unit) la apăsarea butonului dedicat. Acest test implică calcule cu zecimale, fiind mai solicitant pentru procesor.

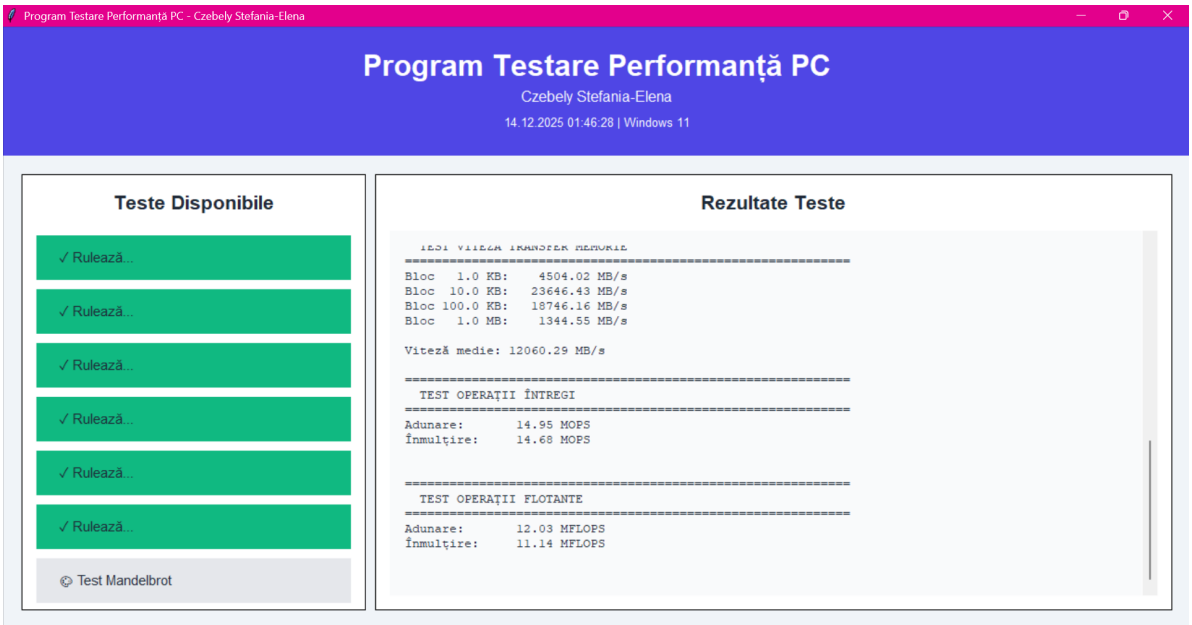


Figura 5.7: Rezultatele testului de Operații Flotante (mijloc).

5.8 5.8. Test Mandelbrot

Ultimul test generează fractalul Mandelbrot. Acesta este cel mai intensiv test computațional, măsurând capacitatea procesorului de a efectua calcule complexe iterative într-un timp scurt.



Figura 5.8: Rezultatul testului Mandelbrot (jos).

6. Concluzii și Evaluare

Proiectul a reușit să creeze o suită completă de testare. Pe baza rezultatelor obținute, putem trage următoarele concluzii despre sistemul de test (Intel Core Ultra 5 135U):

6.1 6.1. Analiza Performanței

Sistemul testat demonstrează capabilități solide pentru un procesor de tip ”U” (Ultra-low power), destinat laptopurilor moderne subțiri și ușoare.

1. **Memoria:** Deși sistemul are mult RAM ocupat (88%), viteza de transfer de peste 12 GB/s este excelentă, asigurând o fluiditate bună în aplicații.
2. **Procesorul:** Diferența mică dintre scorul la întregi (15.49 MOPS) și cel la virgulă mobilă (11.22 MOPS) indică o unitate FPU foarte capabilă. În procesoarele mai vechi, diferența era mult mai mare.
3. **Stabilitate:** Aplicația a rulat fără erori și fără blocaje, validând implementarea corectă a mecanismului de `threading`.

6.2 6.2. Direcții de Dezvoltare

Pentru versiunile viitoare, aplicația ar putea fi extinsă cu:

- **Testare GPU:** Folosind biblioteci precum PyCUDA sau OpenCL pentru a testa placa video.
- **Salvare Raport:** Exportarea rezultatelor într-un fișier PDF sau CSV pentru comparații ulterioare.
- **Grafice:** Generarea unor grafice în timp real pentru utilizarea CPU în timpul testelor.

6.3 6.3. Concluzie Finală

Proiectul îndeplinește cu succes cerințele temei, oferind un instrument funcțional, estetic și precis pentru evaluarea performanței unui PC. Utilizarea limbajului Python a permis o dezvoltare rapidă și un cod curat, ușor de întreținut.

Bibliografie

[1] Neagu, M., *Structure of Computer Systems. Course 2. Computer performance and optimality*, Suport de curs UTCN (SSC_course_2_performance.pdf).

[2] Colectiv SSC, *CPU performance monitoring using the Time-Stamp Counter register*, Îndrumător Laborator 2 (lab2.pdf), UTCN.

[3] Bozdog, R., *Structura Sistemelor de Calcul - Documentație Proiect*, Universitatea Tehnică din Cluj-Napoca (Referință bibliografică internă).

[4] Mihai, A., *Documentație Proiect - Structura Sistemelor de Calcul*, Universitatea Tehnică din Cluj-Napoca (Referință bibliografică internă).

[5] Wikipedia Contributors, *CPUID Instruction*, disponibil online la: `https://en.wikipedia.org/wiki/CPUID` (Accesat: Decembrie 2025).

[6] Python Software Foundation, *Python 3.12 Documentation - Threading (Concurrency)*, disponibil online la: `https://docs.python.org/3/library/threading.html`.

[7] Giampaolo Rodola, *Psutil (Process and System Utilities) Documentation*, disponibil online la: `https://psutil.readthedocs.io`.

[8] Lundh, F., *An Introduction to Tkinter (Standard Python GUI)*, disponibil online la: `https://effbot.org/tkinterbook/`.

[9] Matthew Houglum, *Py-Cpuinfo: Getting CPU Info in Python*, disponibil online la: `https://pypi.org/project/py-cpuinfo/`.

Anexa A: Codul Sursă al Aplicației

Mai jos este prezentat codul complet al aplicației, scris în Python 3.12.

```
import time
import platform
import psutil
import sys
import cpuinfo
import random
from datetime import datetime
import tkinter as tk
from tkinter import ttk, scrolledtext
from threading import Thread

class PerformanceTestGUI:
    def __init__(self, root):
        self.root = root
        self.root.title("Program Testare Performanta PC - Czebely Stefania-Elena")
        self.root.geometry("1000x700")
        self.root.configure(bg='#f0f4f8')

        self.results = {}
        self.create_widgets()

    def create_widgets(self):
        # Header
        header_frame = tk.Frame(self.root, bg='#4f46e5', pady=20)
        header_frame.pack(fill='x')

        title = tk.Label(header_frame, text="Program Testare Performanta PC",
                         font=('Arial', 24, 'bold'), bg='#4f46e5', fg='white')
        title.pack()

        subtitle = tk.Label(header_frame, text="Czebely Stefania-Elena",
```

```
                              font=('Arial', 12), bg='#4f46e5', fg='white')
        subtitle.pack()

        date_label = tk.Label(header_frame,
                              text=f"{datetime.now().strftime('%d.%m.%Y %H:%M:%S')} | {platform.system()} {platform.release()}",
                              font=('Arial', 10), bg='#4f46e5', fg='white')
        date_label.pack(pady=5)

        # Main container
        main_container = tk.Frame(self.root, bg='#f0f4f8')
        main_container.pack(fill='both', expand=True, padx=20, pady=20)

        # Left panel - Buttons
        left_panel = tk.Frame(main_container, bg='white', relief='solid', borderwidth=1)
        left_panel.pack(side='left', fill='both', expand=True, padx=(0, 10))

        button_title = tk.Label(left_panel, text="Teste Disponibile",
                                font=('Arial', 16, 'bold'), bg='white', fg='#1f2937')
        button_title.pack(pady=15)

        # Test buttons
        self.buttons = {}

        tests = [
            ("cpu_info", "Informatii Procesor", self.test_cpu_info),
            ("memory_info", "Informatii Memorie", self.test_memory_info),
            ("mem_access", "Test Timp Acces Memorie", self.test_memory_access),
            ("mem_transfer", "Test Viteza Transfer Memorie", self.test_memory_transfer),
            ("int_ops", "Test Operatii Intregi", self.test_integer_operations),
            ("float_ops", "Test Operatii Flotante", self.test_float_operations),
            ("mandelbrot", "Test Mandelbrot", self.test_mandelbrot),
        ]

        for key, text, command in tests:
            btn = tk.Button(left_panel, text=text, command=lambda c=command, k=key: self.run_test(c, k),
                            font=('Arial', 11), bg='#e5e7eb', fg='#1f2937',
```

```
                            relief='flat', cursor='hand2', pady=10, anchor
='w', padx=20)
            btn.pack(fill='x', padx=15, pady=5)
            self.buttons[key] = btn

        # Right panel - Results
        right_panel = tk.Frame(main_container, bg='white', relief='solid',
 borderwidth=1)
        right_panel.pack(side='right', fill='both', expand=True)

        results_title = tk.Label(right_panel, text="Rezultate Teste",
                                 font=('Arial', 16, 'bold'), bg='white',
fg='#1f2937')
        results_title.pack(pady=15)

        # Results text area
        self.results_text = scrolledtext.ScrolledText(right_panel, wrap=tk
.WORD,
                                                      font=('Courier', 10)
,
                                                      bg='#f9fafb', fg='#1
f2937',
                                                      relief='flat', padx
=15, pady=15)
        self.results_text.pack(fill='both', expand=True, padx=15, pady=(0,
 15))

    def run_test(self, test_func, key):
        self.buttons[key].config(bg='#10b981', text=f"Ruleaza...", state='
disabled')
        self.root.update()

        def run():
            test_func()
            self.buttons[key].config(bg='#10b981', text=f"Ruleaza...",
                                     state='normal')

        Thread(target=run, daemon=True).start()

    def add_result(self, title, content):
        self.results_text.insert(tk.END, f"\n{'=' * 60}\n")
        self.results_text.insert(tk.END, f"  {title}\n")
        self.results_text.insert(tk.END, f"{'=' * 60}\n")
        self.results_text.insert(tk.END, content + "\n")
        self.results_text.see(tk.END)

    def test_cpu_info(self):
```

```
106         info = cpuinfo.get_cpu_info()
107         content = f"Producator: {info.get('vendor_id_raw', 'N/A')}\n"
108         content += f"Brand: {info.get('brand_raw', 'N/A')}\n"
109         content += f"Arhitectura: {info.get('arch', 'N/A')}\n"
110         content += f"Numar de biti: {info.get('bits', 'N/A')}\n"
111         content += f"Nuclee fizice: {psutil.cpu_count(logical=False)}\n"
112         content += f"Nuclee logice: {psutil.cpu_count(logical=True)}\n"
113 
114         freq = psutil.cpu_freq()
115         if freq:
116             content += f"Frecventa maxima: {freq.max:.2f} MHz\n"
117             content += f"Frecventa curenta: {freq.current:.2f} MHz\n"
118 
119         self.results['cpu_info'] = info
120         self.add_result("INFORMATII PROCESOR", content)
121 
122     def test_memory_info(self):
123         mem = psutil.virtual_memory()
124         content = f"Memorie totala: {mem.total / (1024 ** 3):.2f} GB\n"
125         content += f"Memorie disponibila: {mem.available / (1024 ** 3):.2f
    } GB\n"
126         content += f"Memorie utilizata: {mem.used / (1024 ** 3):.2f} GB\n"
127         content += f"Procentaj utilizare: {mem.percent}%\n"
128         self.results['memory_info'] = mem
129         self.add_result("INFORMATII MEMORIE", content)
130 
131     def test_memory_access(self):
132         size = 10_000_000
133         test_array = list(range(size))
134 
135         # Acces secvential
136         start = time.perf_counter()
137         total = 0
138         for i in range(0, size, 100):
139             total += test_array[i]
140         elapsed = time.perf_counter() - start
141         seq_time = (elapsed / (size / 100)) * 1_000_000_000
142 
143         # Acces aleatoriu
144         indices = [random.randint(0, size - 1) for _ in range(100_000)]
145         start = time.perf_counter()
146         total = 0
147         for idx in indices:
148             total += test_array[idx]
149         elapsed = time.perf_counter() - start
150         rand_time = (elapsed / len(indices)) * 1_000_000_000
151 
```

```
152         content = f"Timp mediu acces secvential: {seq_time:.2f} ns\n"
153         content += f"Timp mediu acces aleatoriu: {rand_time:.2f} ns\n"
154         self.results['mem_access'] = {'seq': seq_time, 'rand': rand_time}
155         self.add_result("TEST TIMP ACCES MEMORIE", content)
156 
157     def test_memory_transfer(self):
158         sizes = [1024, 10240, 102400, 1024000]
159         speeds = []
160         content = ""
161         for size in sizes:
162             data = bytearray(size)
163             iterations = max(1000, 100_000_000 // size)
164             start = time.perf_counter()
165             for _ in range(iterations):
166                 temp = data[:]
167             elapsed = time.perf_counter() - start
168             total_mb = (size * iterations) / (1024 * 1024)
169             speed = total_mb / elapsed
170             speeds.append(speed)
171             size_str = f"{size / 1024:.1f} KB" if size < 1024000 else f"{
    size / 1024000:.1f} MB"
172             content += f"Bloc {size_str:>8}: {speed:>10.2f} MB/s\n"
173 
174         avg_speed = sum(speeds) / len(speeds)
175         content += f"\nViteza medie: {avg_speed:.2f} MB/s"
176         self.results['mem_transfer'] = avg_speed
177         self.add_result("TEST VITEZA TRANSFER MEMORIE", content)
178 
179     def test_integer_operations(self):
180         iterations = 10_000_000
181         content = ""
182 
183         start = time.perf_counter()
184         result = 0
185         for i in range(iterations):
186             result += i
187         elapsed = time.perf_counter() - start
188         ops = iterations / elapsed
189         content += f"Adunare:    {ops / 1_000_000:>8.2f} MOPS\n"
190 
191         start = time.perf_counter()
192         result = 1
193         for i in range(1, iterations // 100):
194             result = (result * i) % 1_000_000
195         elapsed = time.perf_counter() - start
196         ops = (iterations // 100) / elapsed
197         content += f"Inmultire:  {ops / 1_000_000:>8.2f} MOPS\n"
```

```
198         self.add_result("TEST OPERATII INTREGI", content)
199 
200     def test_float_operations(self):
201         iterations = 1_000_000
202         content = ""
203 
204         start = time.perf_counter()
205         result = 0.0
206         for i in range(iterations):
207             result += float(i) * 0.5
208         elapsed = time.perf_counter() - start
209         ops = iterations / elapsed
210         content += f"Adunare:    {ops / 1_000_000:>8.2f} MFLOPS\n"
211 
212         start = time.perf_counter()
213         result = 1.0
214         for i in range(iterations):
215             result = (result * 1.0000001) % 1000.0
216         elapsed = time.perf_counter() - start
217         ops = iterations / elapsed
218         content += f"Inmultire:  {ops / 1_000_000:>8.2f} MFLOPS\n"
219         self.add_result("TEST OPERATII FLOTANTE", content)
220 
221     def test_mandelbrot(self):
222         def mandelbrot(c, max_iter=100):
223             z = 0
224             for n in range(max_iter):
225                 if abs(z) > 2:
226                     return n
227                 z = z * z + c
228             return max_iter
229 
230         width, height = 400, 400
231         start = time.perf_counter()
232         points = 0
233         for x in range(width):
234             for y in range(height):
235                 c = complex(-2.0 + (x / width) * 3.0, -1.5 + (y / height)
    * 3.0)
236                 mandelbrot(c)
237                 points += 1
238         elapsed = time.perf_counter() - start
239         perf = points / elapsed
240         content = f"Puncte calculate: {points:,}\n"
241         content += f"Timp total: {elapsed:.2f} secunde\n"
242         content += f"Performanta: {perf / 1_000_000:.2f} MP/s\n"
243         self.add_result("TEST MANDELBROT", content)
```

```
244
245 def main():
246     root = tk.Tk()
247     app = PerformanceTestGUI(root)
248     root.mainloop()
249
250 if __name__ == "__main__":
251     main()
```